



# Edge AI-powered vibration monitoring system with IEPE sensors for predictive maintenance in industrial machinery

Youness Hakam<sup>1b\*</sup>, Mohamed Lamane<sup>1b</sup>, Mohamed Tabaa

*Multidisciplinary Laboratory of Research and Innovation, Moroccan School of Engineering Sciences (EMSI), Casablanca, Morocco*

## ARTICLE INFO

Editor name: Mohamed Fathy El-Amin Mousa

### Keywords:

Vibration  
Fault diagnosis  
Edge AI  
Monitoring  
IEPE  
Maintenance  
Support vector machines (SVM)  
Random forests  
k-nearest neighbors (k-NN)  
Deep learning

## ABSTRACT

This paper presents an embedded system for diagnosing faults in industrial machines by analyzing vibration signals. The system combines lightweight embedded platforms (ESP32 and NanoPi Neo) with high-resolution IEPE piezoelectric sensors to enable local data acquisition, processing, and classification of mechanical conditions. Machine learning and neural network models are deployed directly at the edge, reducing latency and eliminating the need for cloud-based computation. A full vibration dataset was created for different failure scenarios and made available to the public so that other researchers can use it and reproduce the results. We used and tested a number of classification methods on the NanoPi Neo platform to see how accurate and fast they were. These included Support Vector Machines (SVM), Random Forests, k-Nearest Neighbors (k-NN), and neural networks. The results show that classical machine learning techniques are better than deep learning models in both inference time and classification accuracy when there are embedded limitations. Random Forest had an F1-score of 99.17% with 1.56 ms inference time and SVM obtained 98.99% with 1.33 ms latency. The deep models took a much longer time to compute. The suggested method shows that AI may be used at the edge for real-time issue identification, which is a cost-effective and scalable way to do predictive maintenance that follows the rules of Industry 4.0.

## Introduction

For the purpose of predictive maintenance of industrial machines, mechanical vibrations carry information that is fundamentally important. The detection of irregularities at an early stage and the prevention of costly failures are both feasible outcomes that can be achieved through continuous vibration monitoring. The identification of these anomalies in vibration signals has been demonstrated to be successful through the utilization of data processing techniques that include pattern recognition methods and deep learning. On the other hand, in many traditional systems, calculations are carried out on remote servers or personal computers, which introduces limits in terms of latency, dependability, and confidentiality. In contrast, installing learning models directly at the edge offers a number of benefits, including a reduction in reliance on network connectivity, the ability to make choices even when offline, a reduction in the amount of time it takes to discover faults, and the guarantee that sensitive data is kept within the immediate vicinity of the industrial site. Industry 4.0, in which industrial Internet of Things and edge artificial intelligence are playing an increasingly important role in monitoring the health of equipment, fits in well with these issues. A comprehensive vibration acquisition and analysis system is shown in this work. It is comprised of an IEPE-type piezoelectric sensor and an associated instrumentation chain, and it is coupled to a tiny embedded platform that is comprised of an ESP32 and a NanoPi Neo. The uniqueness of our method

\* Corresponding author.

E-mail address: [hakamyouness03@gmail.com](mailto:hakamyouness03@gmail.com) (Y. Hakam).

<https://doi.org/10.1016/j.sciaf.2026.e03367>

Received 11 July 2025; Received in revised form 6 April 2026; Accepted 15 April 2026

Available online 17 April 2026

2468-2276/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

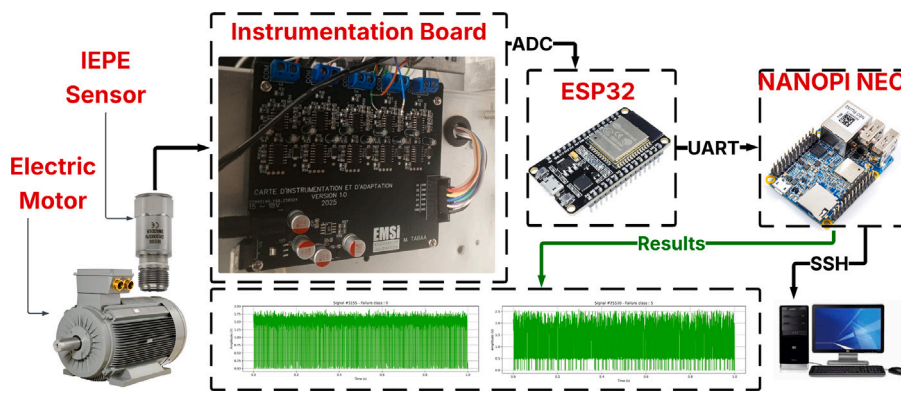


Fig. 1. The architecture of our real-time system.

resides in the fact that we have deployed and evaluated in situ (on the NanoPi Neo) a number of machine learning (ML) and deep learning (DL) models for the purpose of detecting mechanical issues based on vibration data. The database of vibration signals that were gathered has been made available to the public on Kaggle (the dataset is titled “Vibration Dataset of CCS/IEPE sensor AS-062”) in order to encourage repeatability and sharing. Notable contributions to this article include the following: With the design of a comprehensive hardware chain for vibration measurement, a high-precision IEPE industrial sensor is combined with on-board devices (ESP32, NanoPi Neo) for the purpose of data gathering and transmission. In order to serve as a reference in the field, the creation of an annotated database of vibration signals that is made available to the community (through Kaggle) is being undertaken. The implementation and comparison of a number of machine learning and deep learning algorithms directly on the NanoPi Neo allows for the evaluation of the practicability and effectiveness of on-board diagnostics. Fig. 1 demonstrates the architecture of our real-time system. The results that were obtained are discussed, a comparison is made between the effectiveness of deep models and classical models in an embedded setting, and the advantages, limits, and potential for improvement of edge computing in predictive maintenance are highlighted. Last but not least, we will discuss the structure of the article. The following section will provide an outline of the work that is linked to this topic. From the purchasing of hardware until the deployment of the model, the recommended technique is broken down in Section “System Design and Implementation”. Section “Deep Learning Models” presents the outcomes of the experiments, and Section “Results and Discussion” continues with a discussion of those results. The study is brought to a close in Section “Conclusion and Future Work”, which also includes a discussion of potential avenues for further investigation.

## Related work

In recent years, there has been a significant interest in creating smart vibration-based problem diagnosis systems for industrial machinery. This is especially true since edge AI and TinyML frameworks have become popular. These methods represent a move away from centralized cloud-based processing and toward decentralized, real-time analysis that happens directly on embedded devices. Recent research has shown that adding machine learning to low-power microcontrollers like the ESP32 or STM32 family can help find faults quickly and with little use of resources [1,2]. For instance, came up with a real-time bearing fault detection system that runs on an STM32 microcontroller and uses a convolutional neural network (CNN). It has an accuracy rate of over 98% and an inference time of less than 20 ms [3].

In edge-constrained environments, though, classical machine learning methods like Support Vector Machines (SVM), Random Forests, and k-Nearest Neighbors (k-NN) are often better because they are simple, fast, and do not need a lot of memory. They also often outperform deep learning models in terms of computational efficiency and interpretability [4,5]. Hybrid models that use classical classifiers together with feature extraction methods like wavelet transforms, empirical mode decomposition (EMD), and fast Fourier transform (FFT) have also shown good results in terms of robustness and accuracy of classification [6,7].

Another noteworthy development is the employment of high-fidelity IEPE piezoelectric sensors with specialized data acquisition (DAQ) boards made for edge applications. These sensors make it possible to monitor vibrations at a high resolution, which is important for finding little changes in how a machine works. Authors in [7,8] came up with fully programmable DAQ systems that can gather and preprocess IEPE sensor inputs in real time. These systems can do FFT and feature extraction on board without needing external processing units. In a similar way, researchers have looked into how to combine Analog Devices’ CN0540 DAQ board with ESP32-S3 microcontrollers to make it easier to collect vibration data in the field [9].

Several public datasets that record vibration signals under different fault conditions have been made available to encourage reproducibility and benchmarking. These include the FAN-COIL-I dataset [9], the CCS/IEPE motor vibration dataset [10], and the Case Western Reserve University Bearing Dataset [11]. The last one is still a widely used benchmark for diagnosing bearing faults. further datasets, such the IMS bearing dataset [12] and the MIMII dataset for machine operating sounds and vibrations [13], have helped train and test embedded models even further.

Adaptive learning methods have also been added so that embedded models can retrain on the device, which solves the problem of data drift in changing contexts [14]. Neuromorphic computing has become another low-power option for edge-based fault detection.

Spiking neural networks (SNNs) have biologically inspired topologies that work well for low-latency streaming applications [15]. Even though they have potential, these models are still in the early stages of study and have trouble being used in industrial settings.

Recent research has also looked into using federated learning frameworks that let several edge devices work together to train problem diagnostic models without sharing raw vibration data. This keeps data private while increasing generalization [13]. Other research has looked at quantization and pruning methods to make models smaller and easier to operate, and they have worked on cheap hardware in real time [16].

Thorough evaluations that show how important it is to find a balance between algorithmic complexity and the limitations of embedded systems. This is especially true when using multi-stage pipelines that include acquisition, preprocessing, and classification. Finally, new low-cost ways to get high-frequency signals keep coming out. For example, Soto-Ocampo et al. [17] came up with an embedded system that can pick up quick vibration signals from rotating machinery utilizing cheap parts.

All of these research show that combining embedded AI, high-resolution sensors, and strong machine learning methods could lead to the development of scalable, energy-efficient, and real-time diagnostic tools for predictive maintenance in Industry 4.0 ecosystems.

## System design and implementation

This part gives a full picture of the suggested fault diagnosis system, including the hardware parts and the plan on how to install the software. The description is in the order that steps performed during the development and implementation phase. First, the hardware setup and system architecture are shown, with a focus on how high-sensitivity IEPE piezoelectric accelerometers may be used with small embedded platforms like the ESP32 microcontroller and the NanoPi Neo board. These parts were chosen because they have a good blend of computing power and low power use, which makes them great for real-time edge computing applications.

After a brief description of the gear, the procedure of collecting data is explained in depth. We used IEPE sensors to gather vibration signals under different operating and fault conditions. We sampled the signals at a high frequency to keep the dynamic properties of mechanical behavior. The data that was collected was put into a structured database that was organized by fault type, operational mode, and time stamp. This made it easier to find and utilize again in the future. The database was built to be able to grow, and it is open to the public so that people can use it again and work together to make it better.

Then, signal pre-processing methods were used to improve the quality of the data and find useful features. These included bandpass filtering to reduce noise, normalization to change the amplitude scales, and windowing to break up continuous signals into fixed-length samples that machine learning models may use. We took out information from both the time domain and the frequency domain, which let the models capture both the short-term and long-term patterns of vibration. Finally, this paper talks about the learning models that were tested. We used and tested a number of classifiers on the NanoPi Neo platform, including Support Vector Machines (SVM), Random Forest, k-Nearest Neighbors (k-NN), and a lightweight fully connected neural network. We carefully chose model parameters as the kernel type for SVM, the number of trees for Random Forest, and the architectural depth for the neural network to get the best performance possible given the limitations of the embedded hardware. We looked at each model's accuracy in classifying, how long it took to make an inference, and how much memory it used. This gave us a clear idea of how well they would work for real-time edge-based fault diagnostics.

### Hardware and system architecture

The implemented system is composed of several integrated hardware components designed to perform vibration-based fault diagnosis in industrial settings, with a focus on efficient signal acquisition, conditioning, and local processing at the edge. The primary components include an IEPE piezoelectric accelerometer (model AS-062), an IEPE signal conditioning board, an ESP32 microcontroller for real-time analog signal acquisition, and a NanoPi Neo single-board computer responsible for data storage, remote access, and execution of machine learning models. The AS-062 sensor is a high-sensitivity IEPE accelerometer, providing a sensitivity of 100 mV/g and an operational frequency bandwidth ranging from 1.5 Hz to 13 kHz. As a constant-current type sensor, it requires a 4 mA excitation current and outputs a signal composed of an AC vibration component superimposed on a DC bias voltage of approximately 12.55 V. To condition this signal for digital acquisition, an IEPE instrumentation board was designed to fulfill three key functions:

- (i) provide the constant current excitation required by the sensor,
- (ii) decouple the AC vibration signal from the DC bias via a capacitive circuit,
- (iii) amplify and filter the signal to match the input range of the analog-to-digital converter (ADC). An anti-aliasing filter is included in this stage to preserve signal fidelity and prevent spectral distortion during digitization. The conditioned analog signal is digitized in real time using a 12-bit integrated on the IEPE board and interfaced with the ESP32 microcontroller via an SPI connection. The ADC supports sampling rates up to 10 kHz, which is sufficient for most industrial vibration diagnostics applications. The ESP32, operating in real-time acquisition mode, packages the digitized data and streams it continuously through a UART serial link to the NanoPi Neo for further processing. The NanoPi Neo, a compact and low-power ARM-based microcomputer running the Armbian Linux distribution, serves as the central processing node. It receives the incoming vibration data via UART, manages data storage in both .npy file format and an optional local database, and acts as a remote interface for monitoring and control via SSH. This board also hosts and executes Python-based scripts for signal processing and inference of machine learning (ML) and deep learning (DL) models. A management layer implemented as a shell or Python script on the NanoPi controls the acquisition workflow, including sending start/stop commands to the ESP32 and configuring parameters such as the sampling rate and data length via serial communication. Overall, the system provides a low-cost, scalable, and fully embedded solution for continuous vibration monitoring and intelligent fault diagnosis. Fig. 1 presents the complete system architecture, from analog signal acquisition at the sensor to digital signal analysis and user access over the network (see Fig. 2).

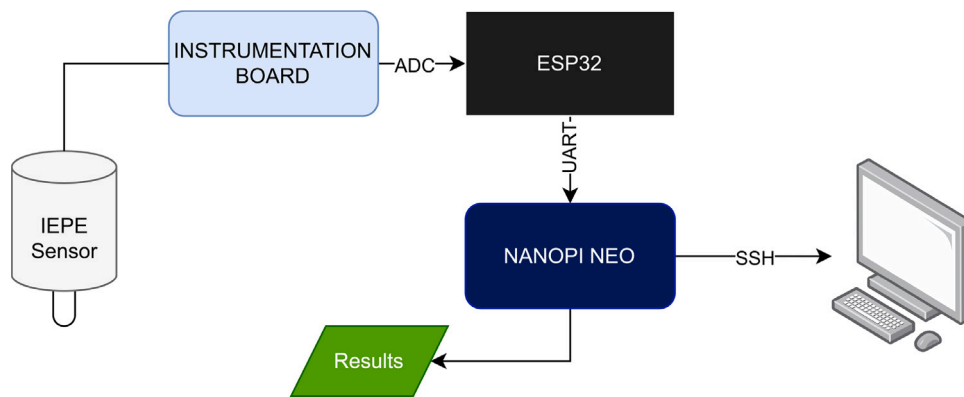


Fig. 2. Schematic diagram of the vibration measurement system architecture.

### Data acquisition

The device collects acceleration data at a sample frequency of about 10 kHz in order to capture the mechanical behavior of spinning machinery and find typical vibration patterns. We chose this rate because it covers the important vibration spectrum well, especially the frequency range of interest, which in this case study goes from about 20 Hz to 5 kHz. This range covers both the basic rotational frequencies and the harmonics that go along with them. These are often seen in industrial fault scenarios.

The system was set up on a special test bench that included an electric motor connected to a mechanical load. The bench also had ways to add different controlled mechanical defects. These problems were meant to mimic common ways that things fail in the real world, like being out of balance, not being aligned properly, being too loose, or having bad bearings. The IEPE sensor was firmly attached to the machine frame to make sure that the signal was sent reliably and to reduce artifacts that could happen during mounting. Fig. 3 presents the steps involved in the signal processing workflow. The system took many time-domain recordings for each operational condition, and each one lasted a few seconds. The ESP32's built-in ADC turned the sensor's analog vibration signal into digital data and sent it to the NanoPi Neo in real time over a UART serial connection. When the NanoPi Neo got the data, it gave each segment a time stamp and saved the signal as either raw .npy files or in a structured database on the device. The whole measurement effort created a labeled dataset with six different operating classes: one for the normal (healthy) condition and five for different kinds of mechanical problems. There are about 10,000 samples per channel in each recording, which is about 1 s of vibration data. There are two columns in each sample: time and acceleration. There is also information that tells you the class label for that sample. A qualitative look at the signals shows that the faulty classes usually have higher amplitude and signal energy than the healthy class. This is consistent with the presence of mechanical problems. The full dataset is available to the public on Kaggle to encourage repeatability and make it easier for future research. This database, which is free to the public, is a useful tool for comparing vibration-based defect detection algorithms in embedded systems.

### Signal preprocessing

Before using classification methods, the raw vibration signals go through a number of preprocessing stages that are meant to make the signals better, make feature extraction easier, and make sure they work with the machine learning pipeline. The preprocessing method is meant to be easy on the computer, so all operations may be done directly on the NanoPi Neo platform without needing any outside processing power.

The first step is to get rid of the DC offset. This is because IEPE sensors usually send a signal that is a high-frequency AC vibration on top of a big DC bias voltage. The IEPE conditioning board has a hardware high-pass filter to get rid of this bias, but a second digital high-pass filter with a cutoff frequency of 10 Hz is used in software to get rid of any low-frequency drift or environmental noise that is still there.

Next, the signals are set to a predetermined range of amplitudes, which is usually done by making each signal have a mean of zero and a variance of one. This stage makes sure that all of the inputs to the machine learning models are on the same scale. This stops the learning algorithms from favoring signals with larger amplitude because they are more severe.

The system uses a sliding window method to divide each signal into fixed-length pieces of 1 s (10,000 samples) so that we may get consistent input samples. If you want to add more training data without taking more measurements, you can use overlapping windows. Each window is seen as a separate sample and given a label based on the type of fault state it represents.

The Fast Fourier Transform (FFT) is also used to change the frequency domain so that the spectral content of the vibration signals can be studied. In many cases of fault diagnosis, frequency-domain properties including peak amplitudes, harmonics, and spectral energy distribution give important information about what kind of problems there are and where they are. You can combine spectral features with time-domain statistics (such as RMS, kurtosis, skewness, and crest factor) to make a rich feature vector that helps with classification.

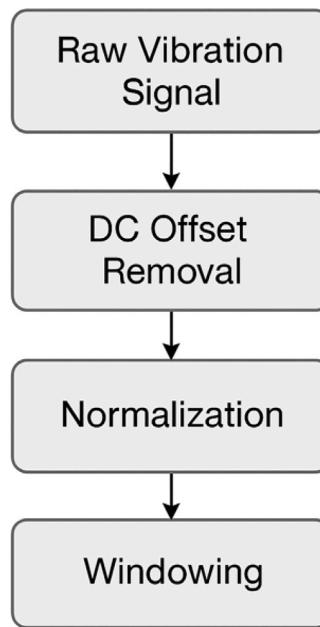


Fig. 3. The steps involved in the signal processing workflow.

The system uses fast NumPy and SciPy routines in Python to do all of the preprocessing stages. These methods are tuned to operate on the NanoPi Neo with embedded Linux. The features that come out of this process, which can be either raw time sequences or derived statistical or spectral descriptors, are then used as input for the classification models that will be discussed in the next section.

#### Machine learning models and implementation

We created and tested a number of machine learning models on the NanoPi Neo platform to find out what different operational scenarios the monitored machine could be in, both healthy and broken. The goal was to check how accurate their classifications were and see if they could be used in real time in an embedded setting with limited computing power. Support Vector Machines (SVM), Random Forest, k-Nearest Neighbors (k-NN), and a light neural network are some of the models chosen for this investigation. These algorithms are well-known in the field of fault detection and are great for use in embedded systems because they do not need a lot of memory or computing power. The input for the standard machine learning models was feature vectors created from the preprocessed vibration signals. These features included both time-domain statistics (such root mean square (RMS), skewness, kurtosis, and crest factor) and frequency-domain characteristics (including spectral centroid, peak frequency, and energy distribution). The proposed system uses a stratified split to make sure that each class was fairly represented in the training and test datasets. All of the characteristics were normalized and put into these datasets. we set up the SVM classifier with a radial basis function (RBF) kernel, which is recognized for doing well with situations that are not linear. We improved the Random Forest model by changing the number of decision trees to find a reasonable balance between speed and performance. We evaluated different values of the neighborhood parameter  $k$  for the k-NN model to see how they affected the classification results. We also made a tiny neural network with a single hidden layer that used ReLU activation and a softmax output layer. This was done to make the architecture basic enough for real-time inference on the NanoPi Neo. the proposed system used the scikit-learn and TensorFlow Lite packages in Python to create all of the models. Inference was done on the NanoPi Neo, which runs a lightweight version of Linux called Armbian. We looked at three main things to rate each model: how accurate it was at classifying data, how long it took to make a decision for each sample, and how much memory it used. The results showed that traditional machine learning models, especially SVM and Random Forest, were more accurate and made decisions faster than the neural network. This makes them better for use in embedded diagnostic systems that do not have access to cloud computing or high-end hardware.

These results show that it is possible to use intelligent vibration-based defect identification at the edge. Low-cost embedded systems can do real-time condition monitoring in line with the ideas of predictive maintenance and Industry 4.0 if the right model is chosen and optimized.

#### Deep learning models

Several vibration classification models have been constructed and assessed, including both “classical” machine learning and deep learning approaches. These models include: A Random Forest with one hundred trees, a Support Vector Machine (SVM) with

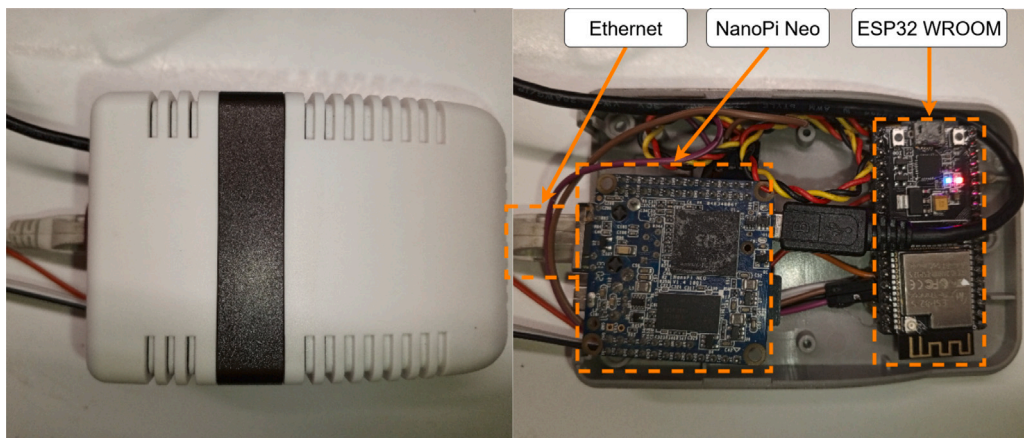


Fig. 4. Internal view of the embedded system showing the NanoPi Neo, ESP32 WROOM, and Ethernet interface.

an RBF kernel, and a k-Nearest Neighbors (k-NN) algorithms were among the supervised algorithms that we put through their paces. The models in question are based on feature vectors that were extracted in the past. With the help of cross-validation, their hyperparameters were optimized, such as the number of neighbors for the k-nearest neighbors algorithm or the C and  $\gamma$  parameters for the support vector machine. The processing required by these techniques is relatively low, and they can be executed on the NanoPi Neo without a great deal of difficulty. This is especially true once the model has been trained, as the inference of a tree or SVM continues to be executed quickly. For the purpose of end-to-end categorization of vibration segments, we have trained a one-dimensional convolutional neural network (CNN) as part of our deep learning initiatives. Models that are utilized in vibration diagnostics served as inspiration for the architecture of the CNN. For instance, the CNN has three one-dimensional convolutional layers that are interspersed with pooling layers, followed by two fully-connected layers, which ultimately results in six outputs that are six-class softmax. The model contains tens of thousands of parameters, which is a reasonable number for deployment on a modest ARM CPU. The scale of the model is also reasonable. Raw time sequences of a predetermined length are used as the CNN's inputs. Alternatively, its Fourier spectrum may be included for the purpose of obtaining a frequency representation. The system has also conducted experiments using a straightforward multi-layer perceptron (MLP) on the manual input features in order to determine how even a fundamental neural network compares to the machine learning techniques that were previously stated. The classical models consist of the following: a support vector machine (SVM) with a kernel based on a radial basis function (RBF) and a regularization parameter  $C = 1$ , a kernel normalization network (KNN) with  $k = 5$  and uniform distance weighting, and a decision tree (RF) consisting of 100 decision trees with a maximum depth of 10. Within the realm of deep learning, three different neural architectures were put into action: A convolutional neural network (CNN) with two convolutional layers (32 and 64 filters, kernel size of 5), followed by max-pooling, dropout, and two fully connected layers; an artificial neural network (ANN) with three dense layers of sizes [128, 64, 32] using ReLU activation functions; a recurrent neural network (RNN) based on two stacked unidirectional LSTM layers (64 units each), followed by a dense output layer. Model training, letter E Using a dataset, the models were trained offline before being used. For traditional machine learning models, training consists primarily of modifying the parameters on the collection of calculated features, computing centroids for k-nearest neighbors, and constructing trees for constrained features. A subset of the data, approximately eighty percent, was used for training CNN, while the remaining data was used for validation. The training process was carried out using gradient descent, Adam optimizer, learning rate 0.001, and 100 epochs. After defining and training the network with Tensorflow, the trained model was exported to the NanoPi Neo for inference using TFLite. Tensorflow helped define and train the network. Scripts written in Python are used to control model inference on the NanoPi Neo machine. Using scikit-learn, simple machine learning models such as k-NN, RF, and SVM were deployed. In addition, we utilized TFLite on the NanoPi within the CNN model in order to enhance its performance. Because of its relatively small size, the CNN model is able to be stored in memory. The amount of time required for each segment of its inference is on the order of a few tens of milliseconds when performed on the NanoPi. We assessed inference times and CPU load on the NanoPi for each model, in addition to classification accuracy measures, so that we could compare the performance of each model under embedded conditions.

## Results and discussion

The developed embedded system was successfully deployed for real-time vibration-based fault detection. Its compact design integrates an ESP32 microcontroller for analog signal acquisition and a NanoPi Neo board for signal processing and machine learning inference. As shown in Fig. 4, both components are enclosed in a protective casing with wired Ethernet connectivity, providing a robust and portable solution for industrial environments.

To validate the classification capability, vibration signals were recorded under six conditions: one healthy state and five simulated fault scenarios. The raw time-domain waveforms from each class are illustrated in Figs. 5–10. These plots reveal clear differences in

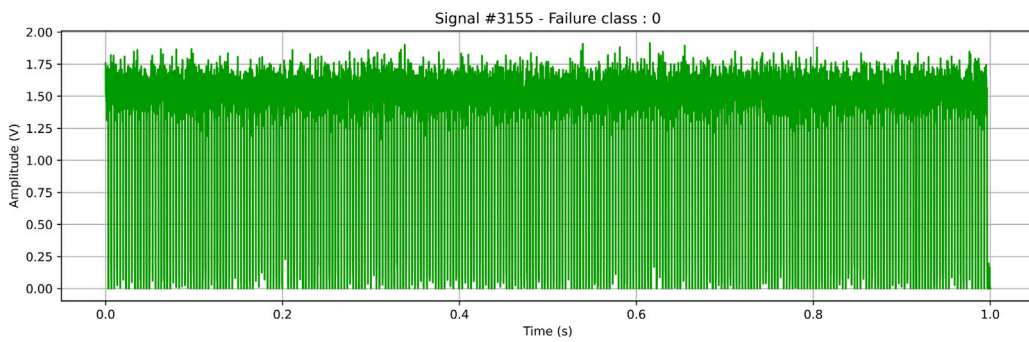


Fig. 5. Time-domain signal for healthy condition (Class 0).

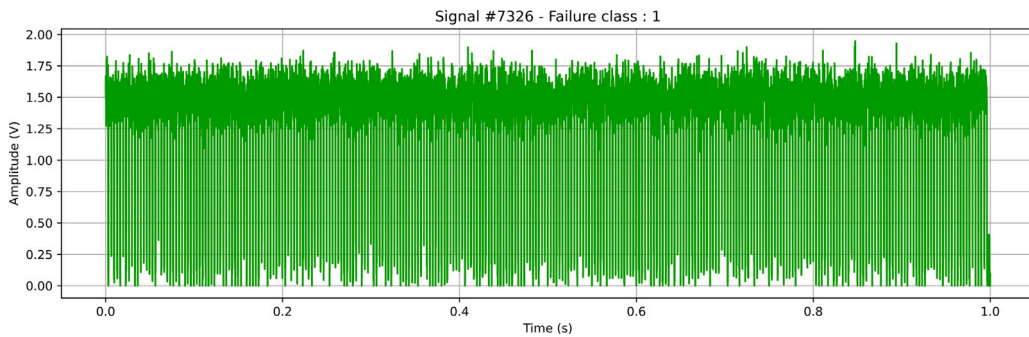


Fig. 6. Time-domain signal for failure class 1.

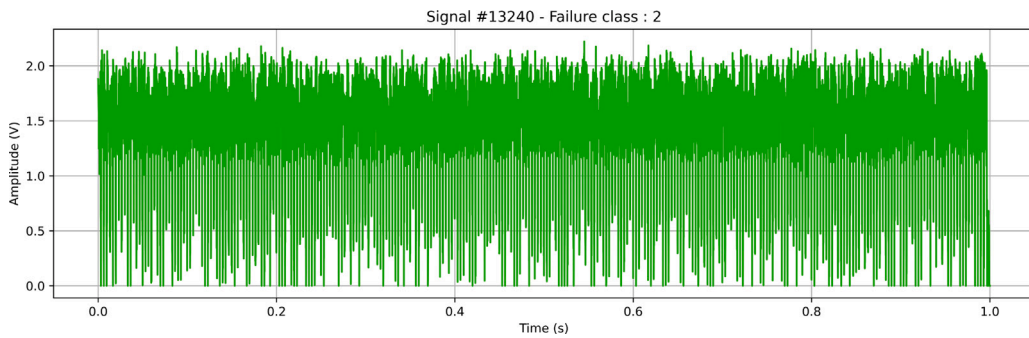


Fig. 7. Time-domain signal for failure class 2.

amplitude, energy, and signal regularity. The healthy signal (Fig. 5) shows low amplitude and stable periodic patterns. In contrast, signals from faulty states (Figs. 6–10) exhibit increased amplitude, irregular bursts, and higher signal energy—common indicators of mechanical defects.

In addition to time-domain inspection, the vibration signals were also analyzed in the frequency domain using short-time Fourier transforms. The spectrogram shown in Fig. 11 highlights the presence of dominant frequencies below 2 kHz, consistent with the expected vibration signatures of rotating machinery. These spectral features play an important role in distinguishing between fault types that may appear similar in the time domain. Several machine learning models were evaluated on the NanoPi Neo using the collected dataset. The performance metrics are presented in Table 1. Traditional models such as Support Vector Machines (SVM) and Random Forest (RF) achieved the highest F1-scores, exceeding 99%, with inference times under 2 ms. This makes them ideal for deployment in constrained edge environments. Deep learning models such as CNN, ANN, and RNN offered competitive accuracy but required significantly more computational time, making them less practical for real-time execution in their current form. Table 1 shows the cross-validated performance of all compared models on the CCS/IEPE AS-062 dataset. Experimental results, reported the performance in terms of accuracy averaged across stratified 5-fold cross-validation for statistical significance and reproducibility. In each fold, the class balance is kept and 80% of data are training sets whereas 20% are testing sets. Classical machine learning models outperform on-embedded solutions. Notably, the random forest and support vector machine classifier

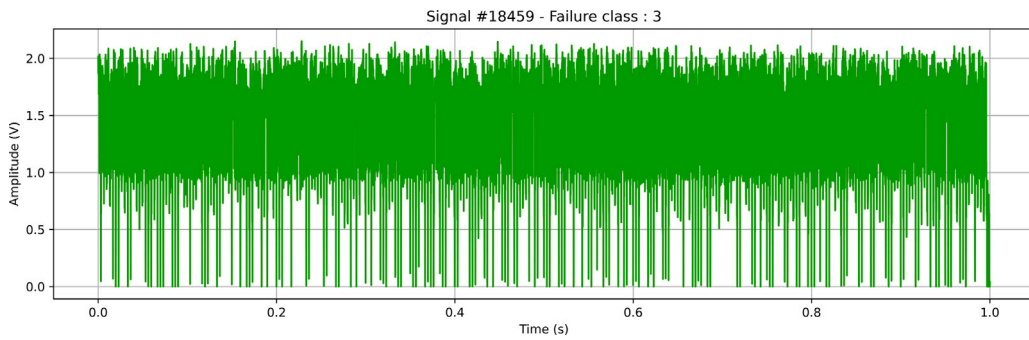


Fig. 8. Time-domain signal for failure class 3.

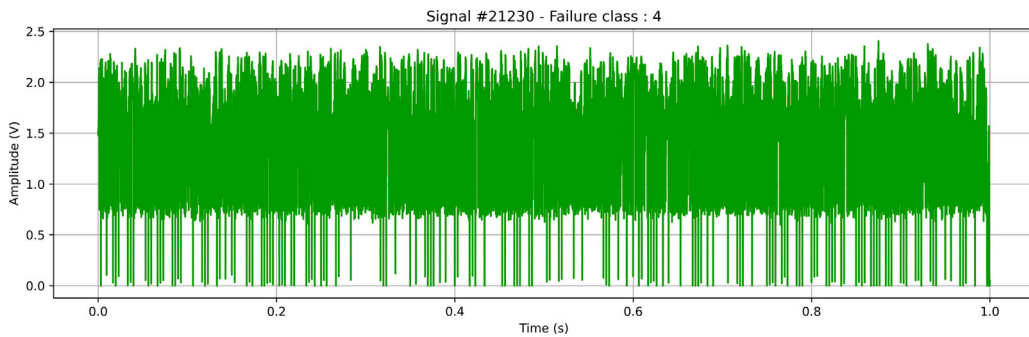


Fig. 9. Time-domain signal for failure class 4.

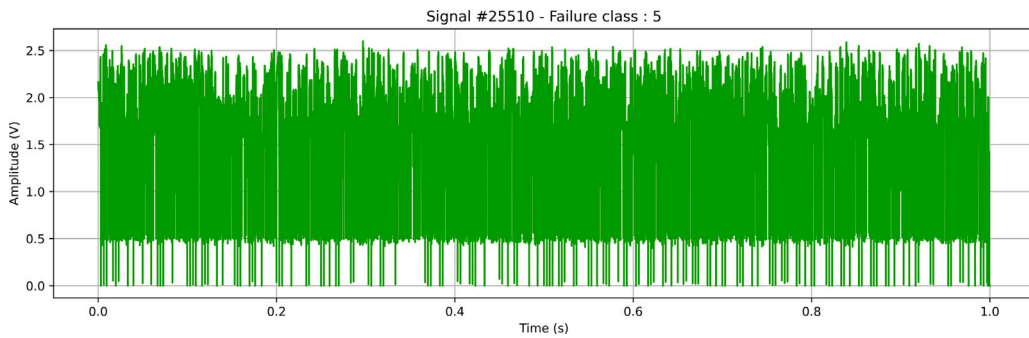
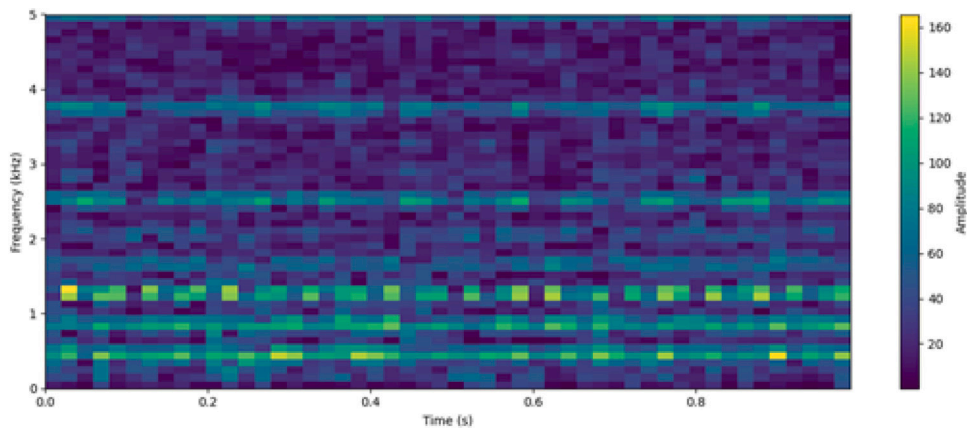


Fig. 10. Time-domain signal for failure class 5.

provided the highest F1-scores of  $99.17 \pm 0.15\%$  and  $98.99 \pm 0.20\%$ , respectively. The small standard deviation values show stable and similar behavior across folds, validating the robustness of these models on balanced six-class dataset. On the other hand, SS and DS network architectures such as CNN, ANN and RNN reported lower F1-scores with a large standard deviation across folds. The RNN achieved an accuracy of  $97.53 \pm 0.56\%$  but with much longer and flexible inference time than classical methods. For speed comparison, the inference time was run on a NanoPi Neo device—with results averaged for 1000 trials and presented as means  $\pm$  standard deviation. We note that for all models, the processing time is well within 2 ms (average across multiple runs) demonstrating the potential for real-time edge deployment of Classical ML models. Deep learning models on the other hand took a time ranging between 12 ms–15 ms for each inference, which made them less preferable for embedded applications that were latency-critical. In conclusion, the experiments demonstrate that classical machine learning models can achieve the optimal compromise between classification accuracy, stability and computational overhead for large-scale vibration classification with limited computing resources. To provide context for these results, Table 2 compares the performance of the current system with similar works reported in the literature. The SVM and RF implementations in this work outperform several more complex models (e.g., CNN, CNN-LSTM) used on more powerful hardware such as the Jetson Nano and Raspberry Pi 4. The low inference times and high classification accuracy observed in this study demonstrate that classical machine learning models, when properly optimized, can be both efficient and reliable for embedded condition monitoring.



**Fig. 11.** Spectrogram of a fault signal showing energy distribution over time and frequency.

**Table 1**

Cross-validated performance on the CCS/IEPE AS-062 dataset (Mean  $\pm$  Standard Deviation).

| Model | Precision (%)    | Recall (%)       | F1-score (%)     | Inference time (ms) |
|-------|------------------|------------------|------------------|---------------------|
| SVM   | 99.03 $\pm$ 0.18 | 98.97 $\pm$ 0.21 | 98.99 $\pm$ 0.20 | 1.33 $\pm$ 0.05     |
| RF    | 99.19 $\pm$ 0.14 | 99.14 $\pm$ 0.17 | 99.17 $\pm$ 0.15 | 1.56 $\pm$ 0.07     |
| k-NN  | 98.79 $\pm$ 0.25 | 98.71 $\pm$ 0.28 | 98.75 $\pm$ 0.26 | 1.84 $\pm$ 0.06     |
| CNN   | 93.51 $\pm$ 0.82 | 92.81 $\pm$ 0.91 | 93.16 $\pm$ 0.86 | 14.78 $\pm$ 0.45    |
| ANN   | 96.00 $\pm$ 0.63 | 95.65 $\pm$ 0.71 | 95.83 $\pm$ 0.67 | 12.21 $\pm$ 0.39    |
| RNN   | 97.57 $\pm$ 0.54 | 97.48 $\pm$ 0.58 | 97.53 $\pm$ 0.56 | 15.32 $\pm$ 0.48    |

**Table 2**

Comparison with other embedded fault diagnosis approaches.

| Reference       | Platform       | Model         | Accuracy (%) | Time (ms) |
|-----------------|----------------|---------------|--------------|-----------|
| Ref [1]         | STM32F746      | CNN           | 94.50        | 17.20     |
| Ref [18]        | Raspberry Pi 4 | SVM           | 95.60        | 12.50     |
| Ref [19]        | ESP32          | k-NN          | 93.80        | 3.60      |
| Ref [20]        | Jetson Nano    | CNN-LSTM      | 96.40        | 20.10     |
| Ref [21]        | STM32F407      | CNN           | 94.00        | 15.80     |
| This work (SVM) | NanoPi Neo     | SVM           | 98.99        | 1.33      |
| This work (RF)  | NanoPi Neo     | Random Forest | 99.17        | 1.56      |

**Table 3**

Latency breakdown of the embedded processing pipeline on NanoPi Neo.

| Processing stage                                           | Mean time (ms) | Standard deviation (ms) |
|------------------------------------------------------------|----------------|-------------------------|
| Signal preprocessing (high-pass filtering + normalization) | 0.41           | 0.03                    |
| Feature extraction (FFT + statistical features)            | 0.37           | 0.02                    |
| Model inference (SVM classifier)                           | 1.33           | 0.05                    |
| <b>Total end-to-end latency</b>                            | <b>2.11</b>    | <b>0.07</b>             |

In order to gain a better insight into the computational burden of every component in our proposed embedded diagnostic pipeline, we performed a detailed timing analysis. We decomposed the total execution latency into three components; preprocessing, feature extraction and inference. All timing measurements are done directly on the NanoPi Neo platform under Armbian Linux. To ensure accurate evaluation, each experiment was repeated 1000 times; the first 50 iterations of each were discarded to avoid initialization and cache warm-up effects. Python's time was used to measure the execution time. that can be tampered at runtime (e.g., CPU frequency could scale up and down), in which case we should use `time.perf_counter()` function on machines that handle this cleanly. The latency breakdown of the entire processing pipeline is shown in Table 2. The outcomes highlighted that feature extraction constitutes a large fraction of the overall latency, whereas inference phase in classical machine learning approaches is very fast, allowing real-time vibration-based fault diagnosis on resource-limited edge-device (see Table 3).

## Conclusion and future work

In the proposed work, a vibration monitoring system comprised an IEPE accelerometer (AS-062), signal conditioning custom hardware, ESP32 acquisition module and NanoPi Neo edge computing board was embedded for fault diagnosis. Our approach

was tested on a large public data set of 30,000 balanced vibration recordings (i.e., 8.33 h) with six operational classes to provide thorough and unbiased performance evaluation. Stratified 5-fold cross-validation was employed to enhance the methodological robustness, and all performance measurements were presented as mean  $\pm$  standard deviation. The results of the experiments show that classical ML models, including RF or SVM attained F1-scores higher than 99%, and with low variance between folds, which proved stable classification performance. The directly ARM-based measured latency of inference time was less than 1.6 ms, which proved that the developed models are appropriate for real-time industry monitoring use case. A latency breakdown further verified that the full processing and inference pipeline is indeed real time operational, not relying on an external cloud service. On the other hand, deep learning approaches demonstrated competitive accuracy but consumed much more computational resources and time in inference appeared under the same embedded circumstances. These results indicate that further developed classical machine-learning methods present the most optimal tradeoff between accuracy, computation time, and hardware prerequisites on edge-AI predictive maintenance systems. Through the synergy of balanced large-scale dataset, strict statistical validation and transparent timing analysis, we present a reproducible and practical solution in accordance with Industry 4.0 demand. One limitation of the present study is that experiments were conducted under controlled laboratory conditions with variable-speed. Future work will be to generalize the system to industrial settings with variable speed and research into adaptive learning approaches that can improve robust behavior in dynamic operations.

### CRediT authorship contribution statement

**Youness Hakam:** Conceptualization, Methodology, Hardware design, Software implementation, Data curation, Formal analysis, Writing – original draft, Visualization. **Mohamed Lamane:** Experimental setup, Data acquisition, Validation, Writing – review & editing. **Mohamed Tabaa:** Supervision, Project administration, Methodological guidance, Writing – review & editing.

### Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

### Acknowledgment

No financial support acknowledgments are required to declare it. All authors reviewed and approved the final manuscript.

### References

- [1] Z. Gao, Z. Jiang, Z. Dong, et al., An edge-deployable TinyML approach enhanced by transfer learning for efficient bearing fault diagnosis, *Sci. China Technol. Sci.* 68 (2025) 2220401, <http://dx.doi.org/10.1007/s11431-025-3072-9>.
- [2] Andreas Theissler, Judith Pérez-Velázquez, Marcel Kettelgerdes, Gordon Elger, *Predictive maintenance enabled by machine learning: use cases and challenges in the automotive industry*, *Reliab. Engrg. Syst. Saf.* 215 (2021).
- [3] S. Arciniegas, D. Rivero, J. Piñan, et al., Iot device for detecting abnormal vibrations in motors using TinyML, *Discov. Internet Things* 5 (41) (2025) <http://dx.doi.org/10.1007/s43926-025-00142-4>.
- [4] C.-C. Chung, Y.-P. Liang, H.-J. Jiang, CNN hardware accelerator for real-time bearing fault diagnosis, *Sensors* 23 (13) (2023) 5897, <http://dx.doi.org/10.3390/s23135897>.
- [5] K. Sunaga, M. Kondo, H. Matsutani, Addressing the gap between training data and deployed environment by on-device learning, *IEEE Micro* 43 (6) (2023) 66–73, <http://dx.doi.org/10.1109/MM.2023.3314711>.
- [6] N. Dennler, G. Haessig, M. Cartiglia, G. Indiveri, Online detection of vibration anomalies using balanced spiking neural networks, in: 2021 IEEE 3rd International Conference on Artificial Intelligence Circuits and Systems (AICAS), Washington DC, DC, USA, 2021, pp. 1–4, <http://dx.doi.org/10.1109/AICAS51828.2021.9458403>.
- [7] R. De Fazio, L. Spongano, A. Messina, P. Visconti, A fully programmable DAQ board of vibrational signals from IEPE sensors: hardware and software design, performance analysis, *Electronics* 13 (7) (2024) 1187, <http://dx.doi.org/10.3390/electronics13071187>.
- [8] Heitor Lifschitz, Gabriel Rocha, Hendrio Bragança, Cláudio Filho, Leandro Okimoto, Allan Amorin, Fábio Cardoso, Vibration sensor dataset for estimating fan coil motor health, *Data in Brief* (ISSN: 2352-3409) 56 (2024) 110866, <http://dx.doi.org/10.1016/j.dib.2024.110866>, <https://www.sciencedirect.com/science/article/pii/S2352340924008308>.
- [9] S.A. Aburakhia, R. Myers, A. Shami, A hybrid method for condition monitoring and fault diagnosis of rolling bearings with low system delay, *IEEE Trans. Instrum. Meas.* 71 (2022) 1–13, <http://dx.doi.org/10.1109/TIM.2022.3198477>.
- [10] Small vibration dataset of CCS/IEPE sensor AS-062, 2025, [Online]. Available: <https://www.kaggle.com/datasets/medlamane/small-vibration-dataset-of-ccsipe-sensor-as-062>. (Accessed 12 June 2025).
- [11] Case Western Reserve University Bearing Data Center, Bearing Fault Data Set, Case School of Engineering. Available: <https://engineering.case.edu/bearingdatacenter/download-data-file>.
- [12] J. Lee, H. Qiu, G. Yu, J. Lin, IMS Bearing Data Set, University of Cincinnati, Intelligent Maintenance Systems (IMS) Center, NASA Ames Prognostics Data Repository, [Online]. Available: <https://ti.arc.nasa.gov/tech/dash/groups/pcoe/prognostic-data-repository/>. (Accessed June 2025).
- [13] H. Purohit, R. Tanabe, T. Ichige, T. Endo, Y. Nikaido, K. Suefusa, Y. Kawaguchi, MIMII dataset: Sound dataset for malfunctioning industrial machine investigation and inspection, 2019, arXiv preprint [arXiv:1909.09347](https://arxiv.org/abs/1909.09347).
- [14] W. Zhang, G. Peng, C. Li, Y. Chen, Z. Zhang, A new deep learning model for fault diagnosis with good anti-noise and domain adaptation ability on raw vibration signals, *Sensors* 17 (2) (2017) 425, <http://dx.doi.org/10.3390/s17020425>.
- [15] H. Qiu, J. Lee, J. Lin, G. Yu, Wavelet filter-based weak signature detection method and its application on rolling element bearing prognostics, *J. Sound Vib.* 289 (4–5) (2006) 1066–1090.
- [16] P. Singh, M.K. Singh, R. Singh, N. Singh, Federated learning: Challenges, methods, and future directions, in: S.P. Yadav, B.S. Bhati, D.P. Mahato, S. Kumar (Eds.), *Federated Learning for IoT Applications*, in: EAI/Springer Innovations in Communication and Computing, Springer, Cham, 2022, [http://dx.doi.org/10.1007/978-3-030-85559-8\\_13](http://dx.doi.org/10.1007/978-3-030-85559-8_13).

- [17] C.R. Soto-Ocampo, J.M. Mera, J.D. Cano-Moreno, J.L. Garcia-Bernardo, Low-cost, high-frequency, data acquisition system for condition monitoring of rotating machinery through vibration analysis-case study, *Sensors* 20 (12) (2020) 3493, <http://dx.doi.org/10.3390/s20123493>.
- [18] Dileep Kumar, Sanaullah Mehran Ujjan, Kapal Dev, Sunder Ali Khawaja, Naveed Anwar Bhatti, Tanweer Hussain, Towards soft real-time fault diagnosis for edge devices in industrial IoT using deep domain adaptation training strategy, *J. Parallel Distrib. Comput.* 160 (2022) 90–99, <http://dx.doi.org/10.1016/j.jpdc.2021.10.005>.
- [19] D.S. Satwalia, H.P. Thethi, A. Dhyani, G.R. Kiran, M. Al-Tae, M.B. Alazzam, Predictive maintenance using machine learning: a case study in manufacturing management, in: 2023 3rd International Conference on Advance Computing and Innovative Technologies in Engineering (ICACITE), Greater Noida, India, 2023, pp. 872–876, <http://dx.doi.org/10.1109/ICACITE57410.2023.10183012>.
- [20] Rui Zhao, Ruqiang Yan, Zhenghua Chen, Kezhi Mao, Peng Wang, Robert X. Gao, Deep learning and its applications to machine health monitoring, *Mech. Syst. Signal Process.* (ISSN: 0888-3270) 115 (2019) 213–237, <http://dx.doi.org/10.1016/j.ymssp.2018.05.050>, <https://www.sciencedirect.com/science/article/pii/S0888327018303108>.
- [21] Z. Zhao, Y. Jiao, A fault diagnosis method for rotating machinery based on cnn with mixed information, *IEEE Trans. Indus. Inform.* 19 (8) (2023) 9091–9101, <http://dx.doi.org/10.1109/TII.2022.3224979>.